Port Sentinel Agent

Review 1 Report

Name Rahul Sudarshan
Registration Number 24BCE3006

Chapter 1: Introduction

- **1.1 Project Overview**

Port Sentinel Agent is an Ubuntu service that accepts normal English requests through a command prompt, identifies the process occupying a network port and can stop a uniquely identified process. For example, “What process is hogging port 8080? Stop it.” is interpreted as a TCP inspection followed by a controlled termination request. Here, hogging means occupying the port; the project does not measure resource consumption.

Traditional command-line tools require users to know command syntax and combine inspection with process management. This project adds a language interpretation layer while keeping execution in explicit application functions. The revised code includes Python packages, a C source file and header, a ctypes binding and a per-user systemd service. All application code runs at process level.

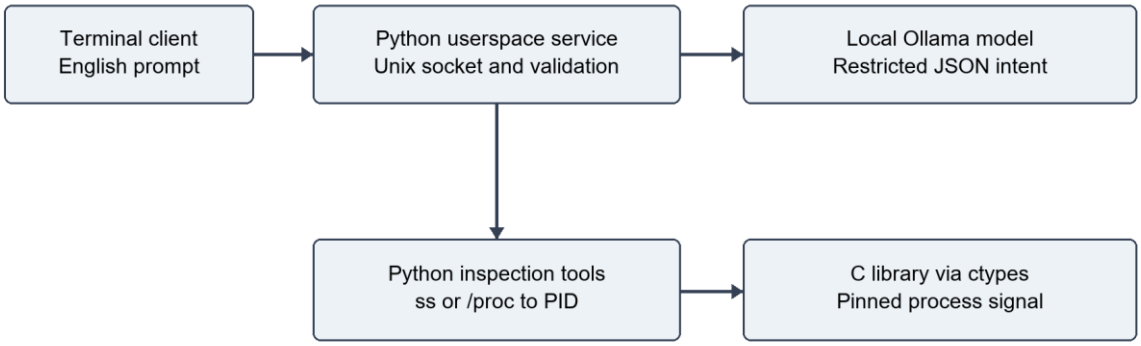


Figure 1. Implemented architecture. A terminal request enters the Python service, the local model returns an intent, and validated Python tools inspect sockets or call the C signalling library. All boxes represent userspace components.

Review status: the original archive did not meet every requirement. The revised source addresses the identified gaps. Local logic tests pass, but live Ubuntu, native build, systemd and real-model validation remain pending. Appendix A records the review evidence.

- **1.2 Problem Statement**

A development server may fail to start because another process is already listening on its port. Resolving the conflict normally requires identifying the endpoint, finding its PID, checking what the process does, and sending an appropriate signal. A wrong PID or an overly broad command can interrupt unrelated work.

The problem is to translate a user's port-management request into a small set of verifiable operations. The language model may misunderstand the port, infer a destructive action from a question or produce invalid output. Socket information may also be incomplete because another user owns the process or the process exits during inspection. The implementation must handle these conditions explicitly.

The supplied prototype used ss and /proc but relied on model instructions for several execution rules. It also lacked service deployment and explicit C integration. The revised design places validation in Python and performs signalling through a narrow C interface rather than executing model-generated shell strings.

- **1.3 Objectives**

Accept command-prompt requests in normal English for inspecting one numeric port, listing ports or stopping a process. Translate the request into a validated action, port, protocol and force flag.

Map TCP listening sockets and UDP bound sockets to visible PIDs using fixed ss arguments, with /proc socket-inode and file-descriptor scanning as a fallback. Preserve IPv4 and IPv6 address information. [1]

Run continuously as an unprivileged per-user Ubuntu service. Provide a terminal client and local Unix socket transport, with no graphical interface or web management endpoint.

Combine Python orchestration with compiled C functions and a header file. Use a pinned Linux process descriptor, recheck identity and ownership, and distinguish signal delivery from observed port release. [2]

Provide repeatable tests, installation instructions, architecture diagrams and review documentation. Explain MCP as background and retain it as a future integration option rather than claim an absent implementation.

The model selects from defined actions only. A stop is refused unless it is explicitly requested, one visible owner is found and the target belongs to the service user. Force termination requires explicit wording; TERM does not automatically escalate to KILL.

- **1.4 Scope and Limitations**

The target platform is Ubuntu 24.04 LTS with Python 3.12, systemd, iproute2, a C compiler and a local Ollama installation. The native library uses Linux pidfd system calls. The project is not a Windows application, kernel module, network sniffer or general-purpose shell. systemd supervises a normal userspace process. [2, 3]

Python uses argparse, socket, socketserver, subprocess, pathlib, ctypes and the Ollama package. The C layer includes port_sentinel.h with errno.h, signal.h, sys/syscall.h and unistd.h. make builds libport_sentinel.so, which ctypes loads with explicit argument types and errno handling.

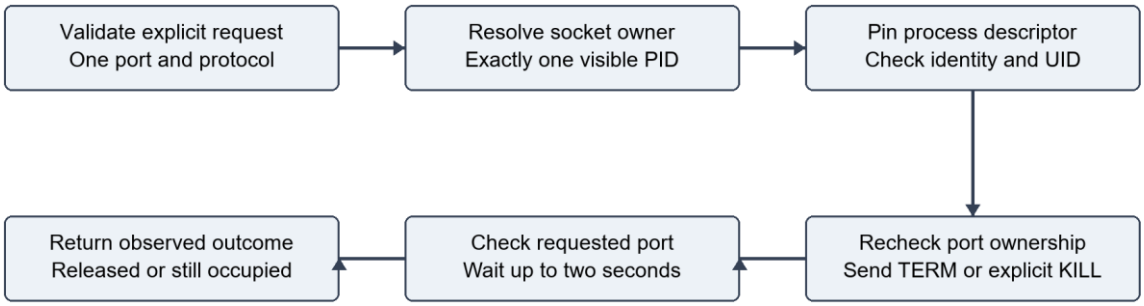


Figure 2. Stop request workflow. Validation and owner discovery precede a pinned process handle, identity and UID checks, a fresh ownership check, signalling and port verification. A failed check returns an error without signalling.

The model receives no administrator privileges. The service runs under the logged-in user, accepts only same-UID peers and refuses other users' processes. Its socket is mode 0600 inside a mode 0700 runtime directory. NoNewPrivileges prevents gaining privileges through execution. A user service follows the login session unless lingering is explicitly enabled. [3]

Requests must contain one numeric port; multi-owner stopping, conditional destructive wording and uncertain requests are refused. TCP defaults to listening sockets. Other network namespaces and TCP established-only connection management are outside scope. Model latency and memory use depend on the selected model and hardware.

A process may ignore TERM or be restarted by another supervisor. The agent reports whether the port is free after a short check. A small socket-state race remains between final ownership validation and signalling; pidfds prevent PID reuse from redirecting the signal but do not freeze socket ownership.

Chapter 2: Literature Review & Theoretical Background

- **2.1 Large Language Models (LLMs) in Code Generation**

Language models can convert varied wording into structured fields or function-call proposals. Ollama documents a tool-calling flow in which the model proposes a call and the application executes it. This separation is useful for system utilities because the application can restrict available operations and validate arguments before touching the operating system. [4]

Port Sentinel narrows this approach further: the model returns a JSON intent containing action, port, protocol and force. Python validates that the port appears in the request, that the protocol matches the wording and that stopping is explicitly requested. Invalid JSON, extra fields, unsupported values and ambiguous ownership cause refusal. No generated code, eval call or arbitrary shell text is executed.

Structured output improves consistency but does not establish that an interpretation is correct. Process names and command lines are treated as data. The final response comes from actual OS results, reducing the chance that a model states a process stopped when only a signal was sent. Real-model accuracy has not been benchmarked in this review.

- **2.2 Introduction to Model Context Protocol (MCP)**

MCP defines a host, clients and servers for exposing tools and context through a common protocol. A client discovers available tools and invokes a server capability using structured requests. A future Port Sentinel adapter could publish inspection and controlled-stop capabilities while retaining the same validation layer. [5]

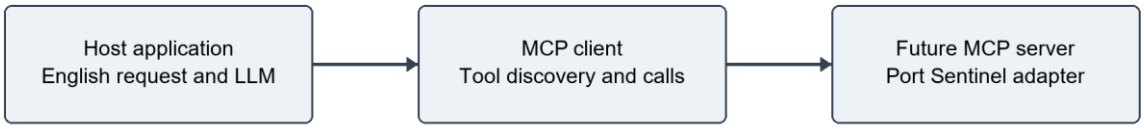


Figure 3. Proposed MCP integration only. A host would use an MCP client to discover and call a future Port Sentinel server adapter. This transport is not included in the current code.

MCP can reduce custom integration work when several hosts need the same tools. It does not itself authorise process termination or make tool results trustworthy. A direct local API is sufficient for this single terminal client; the implemented Unix socket request protocol must not be described as MCP.

- **2.3 Existing Solutions**

The comparison below uses official documentation and the ShellGPT project repository. It compares intended scope and interface, not measured speed or reliability. GitHub's current Copilot CLI is distinguished from the retired GitHub CLI Copilot extension. [6–9]

Solution	Documented approach	Relation to this project
Linux ss and signals	Socket inspection and process signalling through native commands and APIs.	Direct OS foundation; users must interpret results and choose a target. [1, 2]
GitHub Copilot CLI	Terminal assistant with agent capabilities for coding and command-line work.	Broader development scope; Port Sentinel limits execution to port inspection and controlled stopping. [6, 7]
Warp Agent	Natural-language agent accessible in its terminal application and through a terminal CLI.	Broader terminal workflow; this project supplies only a local service and command-prompt client. [8]
ShellGPT	Open-source command-line assistant with shell-command generation and other language-model functions.	A reference for natural-language CLI interaction; this project uses fixed operations rather than generated shell text. [9]
Port Sentinel Agent	Local structured planner, socket/PID tools, Python/C integration and user service.	Focused academic prototype with explicit execution checks; Ubuntu and real-model acceptance remain pending.

The design contribution is the connection between a bounded English intent and an observed OS outcome. Socket discovery establishes a candidate PID, identity checks bind the action to a process instance, and a follow-up inspection reports whether the requested endpoint is still occupied. These checks are narrower than the capabilities of general terminal assistants.

The original archive already separated the agent loop from its OS tools, which provided a useful starting point. The revised source adds the required service and C boundary and moves destructive-action checks out of a system prompt into application code. No comparative benchmark or production-readiness claim is made.

For Review 1, the delivered work establishes the problem, objectives, architecture, theoretical background and implementation direction. The next acceptance milestone is an Ubuntu run that compiles the C library, exercises the service and records real-model requests against disposable servers.

Appendix A Code Review and Validation

All seven files in the supplied archive were inspected. The archive and original template are unchanged. The revised project contains the code, tests and documentation described in this report. The following findings explain why the original could not be certified as meeting every requirement.

Original finding	Correction in revised project
No background service or systemd unit	Added per-user daemon, local client, Unix socket transport and systemd service.
No C source, header or build	Added native C library, header, Makefile and ctypes binding.
Combined ss output column offset	Handle optional Netid column; test TCP, UDP, IPv6 and missing PIDs.
Partial PID enrichment and wildcard proc addresses	Supplement missing owners per row and decode native-order IPv4/IPv6 addresses.
Model could pass an arbitrary PID to kill tool	Validated intent drives execution; discover a unique owner, check UID, identity and ownership.
Numeric PID race and no release check	Pin a pidfd through C, then report signal delivery and observed port status separately.
One fixed-port smoke script	Added regression suite and Linux integration tests using disposable sockets and an ephemeral port.

Validation performed on 20 October 2026: 29 tests discovered, 25 passed and 4 skipped because the host is Windows. Passing tests cover intent checks, negation, force restrictions, protocol and port mismatch, missing or ambiguous ownership, ss parsing, address decoding, timeout handling and partial owner enrichment. Python syntax compilation also passes.

Pending Ubuntu checks: C compilation, TCP discovery and native termination, UDP /proc fallback, native protected-PID rejection, local daemon transport, systemd installation/restart and real-model prompts. The four automated Linux tests are supplied; a separate manual checklist covers the model and installed service. Included CI configuration has not run.

GitHub upload was deferred at the user’s request. The local delivery includes the full revised project and this report. The code review and test log are in docs/CODE_REVIEW.md and docs/test-results-windows.txt.

References

Official documentation and primary project sources consulted on 20 October 2026. References support the design discussion; they do not establish that the revised implementation passed Ubuntu acceptance.

[1] Linux manual pages. ss socket inspection utility.
https://man7.org/linux/man-pages/man8/ss.8.html

[2] Linux manual pages. pidfd_open and process descriptors.
https://man7.org/linux/man-pages/man2/pidfd_open.2.html

[3] systemd documentation. Execution environment, runtime directories and privileges.
https://www.freedesktop.org/software/systemd/man/256/systemd.exec.html

[4] Ollama documentation. Tool calling and application execution.
https://docs.ollama.com/capabilities/tool-calling

[5] Model Context Protocol. Architecture overview in the official repository.
https://github.com/modelcontextprotocol/modelcontextprotocol/blob/main/docs/docs/2026-07-28/learn/architecture.mdx

[6] GitHub Docs. Getting started with GitHub Copilot CLI.
https://docs.github.com/en/copilot/get-started/cli-quickstart

[7] GitHub Docs. Retirement notice for the GitHub CLI Copilot extension.
https://docs.github.com/en/enterprise-cloud%40latest/copilot/how-tos/use-copilot-for-common-tasks/use-copilot-in-the-cli

[8] Warp documentation. Agents overview and terminal CLI access.
https://docs.warp.dev/agents/

[9] ShellGPT. Primary open-source project repository.
https://github.com/TheR1D/shell_gpt

[10] Linux manual pages. Process stat fields and start time.
https://www.man7.org/linux/man-pages/man5/proc_pid_stat.5.html

Implementation note: process start time is read from field 22 of /proc/PID/stat, with the process name parsed separately so spaces or parentheses do not shift field positions. This value is checked after opening the pinned process descriptor. [10]